ADR 0007 — State store

Status: Deferred to v2 (superseded for v1 by in-memory-only decision, 2026-06-23) **Original date**: 2026-06-20 **Downgraded**: 2026-06-23 **Author**: Marv **Related**: ADR 0001, ADR 0002, ADR 0006, docs/DEFERRED.md

---

v1 status (2026-06-23 update)

No persistent state store ships in v1. The SQLite design below is retained as the v2 target, not the v1 implementation.

What v1 actually does:

- Per-request state (`TaskFile`, `action_log`, cost) lives in memory only, scoped to the lifetime of one `Agent.handle_request()` call
- The dispatcher (`FakeAdapterDispatcher`) is constructed fresh per request; fake-adapter state comes from the request's `initial_state` field and is discarded when the response returns
- Idempotency (ADR 0002 §6 "duplicates: return previous report") is NOT enforced across requests in v1 — each request is treated as new
- Cross-run history is NOT tracked by the agent; if Ronin's eval needs to correlate runs it does so from its own `results/*.json` output

Why downgraded: the original ADR was aspirational — no SQLite code shipped in the Phase 1 push. Rather than leave the ADR claiming SQLite while no code exists (dishonest), we're explicit: v1 is in-memory, v2 targets SQLite.

v2 trigger: first Phase 3+ scenario that requires cross-request state — idempotency enforcement, multi-turn sessions with memory, or replay/audit of prior runs. When any of those arrives, implement the schema below.

See DEFERRED.md → "SQLite state store (moved to v2 target, 2026-06-23)" for the trigger and revisit criteria.

Context (original — v2 target)

The agent persists state across the lifetime of a request and across multiple eval runs:

- `task_file` objects for each request (see ADR 0002)
- Action logs (per-tool dispatch results, ordered)
- Cost accumulation per task
- Idempotency tracking (request hashes for duplicate detection per ADR 0002 §6)
- Reversibility state for rollback (per ADR 0009)
- Cross-run history for eval analysis (so a benchmark run can be replayed later)

For v1:

- Single-process, single-machine eval runs
- ~100 scenarios per benchmark run, ~10 tool calls per scenario worst-case → low write volume
- No external services running yet
- Ronin's eval harness reads runs back from disk for analysis

For v3 (potential):

- Multi-process eval runs (sharded scenarios across machines) might need a shared state store
- Higher concurrency might benefit from a process-external store

---

Decision (v2 target — NOT yet implemented)

SQLite for v2. A single `baymax.sqlite` file under `data/runs/<run_id>/baymax.sqlite` per benchmark run. Single connection per process, WAL mode enabled for concurrent reads alongside writes (Ronin's eval harness can read live without blocking the agent).

Redis at v3, conditional. If and only if multi-process eval runs need shared state. Trigger: a concrete v3 requirement that SQLite cannot satisfy. Not pre-emptively migrated.

Schema (v1, evolving — managed via Alembic migrations):

```
runs(run_id PK, started_at, finished_at, backend, model_version, scenario_set_version)
tasks(task_id PK, run_id FK, request_text, status, created_at, completed_at, cost_total,
response_text)
task_files(task_id FK, version, body JSON)               # task_file evolves across recovery passes
action_log(task_id FK, step_idx, tool, arguments JSON, result JSON, started_at, finished_at,
error_type)
idempotency(request_hash PK, task_id, expires_at)
```

Trace data does NOT live here — it goes to jsonl files per ADR 0006. State store is for queryable structured state; traces are for forensic event streams. Separating them prevents one's lifecycle (rollback, expiry) from breaking the other.

Reasoning

[REASONING PROMPTS — expand each bullet into 1-2 sentences in your own voice. Both the v1-in-memory downgrade AND the v2 SQLite target need reasoning here; treat them as two half-decisions.]

On the v1 downgrade to in-memory only (2026-06-23):

- **WHY downgrade rather than ship half a SQLite implementation**: honesty over aspiration. The Phase 1 push didn't ship SQLite code; leaving the ADR claiming SQLite while no code exists is dishonest and would surprise anyone reading the ADR + repo together. Explicit in-memory + v2 target is a truer picture.
- **WHY it's acceptable that v1 loses idempotency + cross-run history + audit**: Phase 1 eval is single-turn, single-process, no idempotency test cases in Ronin's scenario suite. Cross-run history is done by Ronin's eval harness against `results/*.json` output — not by the agent. What v1 loses, v1 didn't need.

On why SQLite is still the v2 target (not Redis, not Postgres):

- **WHY SQLite over Redis for v2**: zero daemon, zero operational surface, transactions built in, WAL mode gives Ronin concurrent read access while the agent writes — same posture as ADR 0006's zero-infra choice. Redis wins on concurrent multi-process writes; v2 doesn't have that need.
- **WHY SQLite over Postgres**: Postgres = a container to run, a connection pool to size, a backup strategy to design. All operational cost for capability v2 doesn't use. Postgres becomes right when you have concurrent writers across processes; single-process eval doesn't.
- **WHY defer Redis to v3 conditional**: not preemptive. Redis migration trigger is a concrete multi-process eval requirement — sharded scenarios across machines, shared session state across instances. Absent that, adopting Redis early = paying operational cost for capability we don't use.
- **WHY separate the state store from the trace store (ADR 0006)**: rollback semantics of state should not corrupt the trace stream. If the state store rolls back a failed transaction, the trace of what led to the failure MUST persist for failure analysis. Coupling them means one's lifecycle breaks the other's forensic value.
- **WHY WAL mode matters specifically**: readers and the writer don't block each other. Ronin's eval harness can watch a live run's state without contention, which is what makes SQLite viable for both write-side (agent) and read-side (eval) simultaneously.

Alternatives considered

Alternative A — Postgres from v1

Run a Postgres container alongside the eval harness; agent + eval both connect to it.

why rejected — Postgres is a container to run, a config to manage, a connection pool to size, and a backup strategy to design. v1 has none of these problems. Postgres becomes correct when we have concurrent writers (multiple agent processes), but v1 doesn't.

Alternative B — Redis from v1

Use Redis (in-memory + AOF persistence) as the v1 state store.

why rejected — Redis is fast and concurrent but loses on durability primitives (transactions across keys are awkward; ACID-grade integrity requires care). And there's nothing v1 needs that SQLite doesn't already give us. Adopting Redis early means paying its operational cost (separate process, RDB/AOF tuning) for capability we don't use. Per RONIN_SYNC.txt #8: deferred to v3 conditional on multi-process need.

Alternative C — File-based JSON / pickle blobs

Write task_files and action logs as JSON files under `data/runs/<run_id>/tasks/<task_id>.json`.

why rejected — no transactions (a crash mid-write leaves a broken JSON file), no cross-task queries (Ronin's eval needs to ask "all tasks where the agent used calendar.create_event" — that's a `SELECT` not a directory scan), and no idempotency primitives. SQLite gives all three for less work than rolling our own.

Alternative D — In-memory only

Keep all state in process memory; lose everything on process exit.

why rejected — eval analysis happens AFTER the run ends. In-memory state means no analysis. Also rules out crash recovery and run resumption.

Consequences

Positive

- Zero new infrastructure in v1 (matches ADR 0006's same posture for traces)
- Transactions: action log writes are atomic with status updates; no inconsistent state on crash
- Queryable: Ronin's eval can `SELECT * FROM action_log WHERE error_type = 'intent_error'` directly
- WAL mode means readers and the writer don't block each other; eval harness can watch a live run
- Backups are `cp baymax.sqlite baymax.sqlite.bak` — trivial

Negative

- Single writer per file — would not scale to multi-process eval (deferred per the Redis trigger)
- Schema migrations are real (Alembic adds a dependency + workflow) — but the alternative is uncontrolled schema drift, which is worse
- SQLite locks the whole file briefly during writes; high concurrency triggers `database is locked` errors. v1 single-writer avoids this.

Neutral

- Storage on disk grows linearly with runs; needs a retention policy by Phase 3 (matches the jsonl traces situation)

Open questions / follow-ups

- **Schema versioning**: managed via Alembic. v1 schema is "Version 1"; any change to the tables above bumps the migration version.
- **Run retention**: not enforced yet. Add a `scripts/retention.py` in Phase 3 to drop old runs.
- **Read amplification**: Ronin's eval queries should be benchmarked against a 200-scenario run by end of Phase 1 to confirm SQLite query time is acceptable.
- **Concurrent eval runs on the same machine**: each run gets its own `baymax.sqlite` file under its own `data/runs/<run_id>/` directory — no cross-run lock contention.
- **Redis migration trigger documented in DEFERRED.md**: multi-process eval runs sharing state.

References

- SQLite WAL mode: https://www.sqlite.org/wal.html
- Alembic: https://alembic.sqlalchemy.org/
- RONIN_SYNC.txt Tier 2 #8 — decision locked

- DEFERRED.md — Redis migration trigger

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.